

中國人民大學

专业硕士学位论文

(中文题目) 面向智能体大语言模型的持续强化学习
Continual Reinforcement Learning
for Agentic Large Language

(英文题目) Models

作者学号: XXXXXXXXXXXX

作者姓名: 孙 豪

所在学院: XXX 学院

专业名称: XXXX

导师姓名: XXX

持续学习; 强化学习; 大语言模

论文主题词: 型; 智能体; 经验回放

论文提交日期: 2026 年 07 月

独创性声明

本人郑重声明：所呈交的论文是我个人在导师的指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得中国人民大学或其他教育机构的学位或证书所使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

论文作者（签名）：_____ 日 期：_____

关于论文使用授权的说明

本人完全了解中国人民大学有关保留、使用学位论文的规定，即：学校有权保留送交论文的复印件，允许论文被查阅和借阅；学校可以公布论文的全部或部分内容，可以采用影印、缩印或其他复制手段保存论文。

论文作者（签名）：_____ 日 期：_____

指导老师（签名）：_____ 日 期：_____

授权书影印件

摘要

线上智能体大语言模型通过周期性的强化学习更新来持续进化：每隔一段时间，用新采集的交互数据精炼策略。这本质上是一个**持续强化学习**问题——模型必须在习得新行为的同时保住已有能力。在智能体、多轮的设定下，两个失败模式使这一问题尤为困难：其一是**灾难性遗忘**，新任务上的更新系统性侵蚀此前已掌握的能力；其二是**Echo Trap**，多轮智能体强化学习中策略多样性坍缩，组内轨迹趋于一致，导致训练停滞或发散。已有强化微调方法虽比监督微调更不易遗忘，但仍然会遗忘，且多轮智能体场景带来了单轮研究无法暴露的失效模式。

针对遗忘的主流解法——经验回放及其各种改进（优先级、分区、压缩缓冲区）——共享一个隐含假设：被回放的与在线的经验携带着**忠实的学习信号**。本文主张，在持续**多轮智能体**强化学习中，这一假设会在任何回放或正则化之前悄然破裂。智能体场景的主力算法 GRPO 通过对同一查询的多条 rollout 在组内归一化来估计每条样本的优势；该估计**仅当**组内轨迹的差异纯粹来自策略采样随机性时才无偏。两个典型的智能体效应在源头破坏了这一前提：一是**组内环境发散**，跨多查询会话的多个执行环境逐渐分叉，使优势吸收随轮次累积的环境噪声；二是**多轮前提漂移**，后续查询通常引用上一轮的执行结果，当其被静态写死而实际状态又是策略相关的随机函数时，前提仅以一个随策略变强而漂移的概率成立，从而奖励对不存在问题的幻觉式修复或惩罚如实回答。

被污染的优势在下游无法修复：任何回放加权或 KL 锚定都救不回一个符号本就错了的梯度。据此，本文把持续智能体强化学习重新表述为一个**两阶段**问题——**先产出干净信号，再巩固它**，并围绕这一主旨组织整套系统。在”把信号做干净”这一半，本文设计了一套沙箱 rollout 系统：每组多个槽位从同一母版派生、起点位级一致，并在每个轮次边界把会话对齐到优胜轨迹，以消除组内环境发散；同时

仅保留真实会话的首条查询作为种子，由观察、出题、奖励三个协作智能体在线构造后续轮次，使每条后续查询的前提由构造保证成立，根除前提漂移，并兼作一个跟随策略能力前沿的自适应课程。在”巩固干净信号”这一半，本文施加一个以零源码改动注入 GRPO 的四项持续学习目标（强化学习、逆向 KL、经验回放、熵正则）；一个按能力而非难度分区、按抗遗忘价值而非奖励排序的七桶优先级回放缓冲区；以及一个 U 形、按动作块的回放损失重加权方案，把信用集中到每条轨迹的早期分支决策与结论段。本文在一个 27B 智能体上跨七个能力域验证整套系统，用 Pass³ 基准同时量化新任务习得与旧任务遗忘，并以受控消融套件隔离每个组件的边际贡献。

关键词：持续学习；强化学习；大语言模型；智能体；经验回放；灾难性遗忘

Abstract

Deployed agentic large language models evolve through periodic reinforcement-learning updates: every so often, the policy is refined with freshly collected interaction data. This is inherently a **continual reinforcement learning** problem—the model must acquire new behaviors while retaining existing capabilities. In the agentic, multi-turn regime, two failure modes make this hard: **catastrophic forgetting**, where updates on new tasks erode previously mastered capabilities, and **Echo Trap**, a collapse of policy diversity in multi-turn agentic RL that drives within-group trajectories toward identical outputs and stalls training. Reinforcement fine-tuning forgets less than supervised fine-tuning, yet it still forgets, and the multi-turn agentic setting introduces failure modes invisible to single-turn studies.

Mainstream anti-forgetting approaches—experience replay and its many variants (prioritized, partitioned, compressed buffers)—share a hidden assumption: that the replayed and online experiences carry a **faithful learning signal**. We argue that in continual **multi-turn agentic** RL this assumption silently breaks, and it breaks **before** any replay or regularization. GRPO, the workhorse algorithm in this setting, estimates a per-sample advantage by normalizing rewards within a group of rollouts of the same query; this estimator is unbiased **only** when the trajectories differ purely from stochastic policy sampling. Two characteristic agentic effects corrupt this premise at the source: **within-group environment divergence**, where the execution environments across a multi-query session gradually fork so that the advantage absorbs environment noise accumulating with the turn index; and **multi-turn premise drift**, where a follow-up query referencing the previous turn’s result is fixed statically while the realized state is a policy-dependent random

function, so the premise holds only with a probability that drifts as the policy improves—rewarding hallucinated fixes to non-existent problems or penalizing honest answers.

A corrupted advantage cannot be repaired downstream: no replay weighting or KL anchoring rescues a gradient whose sign was already wrong. We therefore reframe continual agentic RL as a **two-stage** problem—**produce a clean signal, then consolidate it**—and organize the entire system around this thesis. In the clean-signal half, a sandboxed rollout system spawns each group of slots from a single master template for bit-identical starts and re-aligns the session to the winning trajectory at every turn boundary to eliminate within-group divergence; meanwhile, only the first query of each real session is kept as a seed, and subsequent turns are constructed online by three cooperating agents (observer, questioner, reward), so that every follow-up’s premise is grounded by construction, which also yields an adaptive curriculum tracking the policy’s capability frontier. In the consolidation half, we apply a four-term continual-learning objective injected into GRPO with no framework fork (RL, reverse KL, replay, entropy); a seven-bucket prioritized replay buffer partitioned by capability rather than difficulty and ranked by anti-forgetting value rather than reward; and a U-shaped, action-block-level replay loss reweighting that concentrates credit on the early branching decisions and the concluding segments of each trajectory. We validate the system on a 27B agent across seven capability domains, quantifying both new-task acquisition and old-task forgetting with a Pass³ benchmark, and isolate the marginal contribution of each component through a controlled ablation suite.

Key Words : Continual Learning; Reinforcement Learning; Large Language Model; Agent; Experience Replay

目录

第 1 章 绪论	1
1.1 研究背景	1
1.2 研究问题与动机	2
1.3 主要贡献	2
1.4 论文组织结构	3
第 2 章 相关工作	5
2.1 经验回放与持续强化学习	5
2.2 大语言模型智能体的持续学习与强化学习训练	6
2.3 图形界面智能体的强化学习与持续学习	6
2.4 经典持续学习基础方法	7
2.5 本文与已有工作的关系	7
第 3 章 问题定义	9
3.1 基本设定	9
3.2 GRPO 优势估计的隐含前提	10
3.3 多轮智能体场景的两类信号污染	10
3.3.1 组内环境发散	10
3.3.2 多轮前提漂移	10

3.4 两阶段重新表述	11
第 4 章 方法	13
4.1 持续学习目标	14
4.2 按能力分桶的优先级回放缓冲区	15
4.3 回放的 U 形块重加权	16
4.4 沙箱 Rollout 与 Winner 同步会话	18
4.5 经模拟用户在线构造多轮查询	21
第 5 章 系统实现	25
5.1 基于 verl 的零侵入集成	25
5.2 沙箱架构与厂商无关后端	26
5.3 模型选型	26
5.4 GPU 部署与训练精度	27
5.5 工程解耦与可测试性	28
5.6 当前实现状态与残缺模块	28
第 6 章 实验设计与分析	29
6.1 评测基准	29
6.2 消融实验路线	30
6.3 过程指标与早期预警	31
6.4 预期结果与待回填项	32
第 7 章 总结与展望	33
7.1 工作总结	33
7.2 局限性	34

7.3 未来工作	34
参考文献	37
附录 A 三智能体与奖励模型完整提示词	39
A.1 观察智能体提示词（O6，差异驱动，仅状态）	39
A.2 出题智能体提示词（O3）	40
A.3 奖励模型提示词与判分准则（O4，双通道）	42
致谢	45

图目录

图 4.1 系统总览	14
图 4.2 U 形块权重	18
图 4.3 Winner 同步会话	20
图 4.4 三智能体多轮构造	22

表目录

表 4.1 25k 总容量下各桶平方根配额分配示例 ($q_{min} = 2000$)	16
表 5.1 各角色模型选型 (单一信源: <code>configs/agents.yaml</code>)	27
表 6.1 Phase 3 关键对照轴 (每行只动一个变量)	31

第 1 章 绪论

1.1 研究背景

线上智能体大语言模型的能力来自持续的强化学习后训练：每隔若干周，用新采集的交互数据对策略做一次更新。这构成一个**任务无关的持续强化学习**问题——没有清晰的任务边界，新旧能力在同一参数空间里竞争。与离线训练一次成型不同，持续学习场景下，模型必须在新数据上习得新行为的同时，尽量不遗忘此前已经掌握的能力。

在智能体（agent）、多轮交互的设定下，这一问题比传统持续学习更困难，原因在于两个相互独立的失败模式：

1. **灾难性遗忘（catastrophic forgetting）**：在新任务上的强化学习更新会系统性侵蚀旧领域能力。这是神经网络参数共享带来的固有难题，在持续学习文献中被广泛研究。
2. **Echo Trap^[1]**：多轮智能体强化学习中，策略多样性会发生坍塌——同一查询的组内 rollout 轨迹趋于一致，导致组内归一化所得优势趋近于零，训练停滞乃至发散。这是多轮智能体场景特有的失效模式，单轮强化学习研究中并不暴露。

已有研究表明，强化微调（RFT）比监督微调（SFT）更不易遗忘^[2,3]，但其缓解作用有限，遗忘仍然发生；而多轮智能体场景进一步带来了单轮研究无法暴露的失败模式。因此，如何在多轮智能体大语言模型上做持续强化学习而不发生灾难性遗忘，是一个亟需回答的问题。

1.2 研究问题与动机

针对灾难性遗忘的主流解法是**经验回放**（experience replay）^[4]及其各种改进——优先级缓冲区^[5]、按任务分区缓冲区^[6]、基于核心集的压缩缓冲区^[7]等。这些方法共享一个隐含假设：被回放的与在线的经验携带着**忠实的学习信号**，即用于更新策略的梯度方向是正确的，遗忘只源于“新数据覆盖旧能力”这一参数层面的竞争。

本文主张，在持续**多轮智能体**强化学习中，这个假设会悄悄破裂，而且破裂发生在任何回放或正则化**之前**。智能体场景的主力算法 GRPO（Group Relative Policy Optimization）通过对同一查询的 M 条 rollout 在组内归一化来估计每条样本的优势 A_i 。这个估计**仅当** M 条轨迹的差异纯粹来自策略采样的随机性时才是无偏的。任何其他方差来源都会把环境噪声注入 A_i ，从源头污染学习信号。在多轮智能体场景下，有两个典型效应在源头破坏这一前提：

1. **组内环境发散**。跨多查询会话， M 个执行环境逐渐分叉；某个槽位可能并非因策略更差、而是因前一条查询留下的不利状态被惩罚，于是优势吸收了随轮次累积的环境噪声。
2. **多轮前提漂移**。后续查询通常引用上一轮的执行结果（例如“第 3 页的数据错了”）。当后续查询在数据采集时被写死、而实际状态是策略相关的随机函数时，其前提仅以一个**随策略变强而漂移**的概率成立——于是奖励了对不存在问题的幻觉式修复，或惩罚了如实回答“没有这个问题”。

被污染的优势在下游无法修复：任何回放加权或 KL 锚定都救不回一个符号本就错了的梯度。这一观察把持续智能体强化学习重新表述为一个**两阶段**问题——先产出干净信号，再巩固它——并把本文的整个系统围绕这一主旨来组织。

1.3 主要贡献

在“先产出干净信号、再巩固它”的单一主旨下，本文的贡献如下：

1. **对持续智能体强化学习的源头级重新表述**。本文识别并形式化了 GRPO 学

习信号的两类污染——组内环境发散与多轮前提漂移——它们发生在巩固之前，且无法用回放或正则化修复（第 3 章）。

2. **一套产出干净信号的数据与 rollout 系统。** winner 同步会话调度在多查询会话中恢复组内位级一致起点，以消除组内环境发散；观察、出题、奖励三个协作智能体在线构造前提成立的多轮数据，根除前提漂移，并兼作一个跟随策略能力前沿的自适应课程（第 4 章 §4.4–4.5）。
3. **一套与干净信号匹配的巩固方法。** 以零源码改动注入 GRPO 的四项持续学习目标、按能力分区的抗遗忘回放缓冲区、以及 U 形块级回放重加权方案（第 4 章 §4.1–4.3）。
4. **系统实现与实验设计。** 基于 verl 框架零侵入落地的端到端训练系统（第 5 章），以及在 27B 智能体、七个能力域、Pass³ 评测基准上的逐组件受控消融实验设计（第 6 章）。

关于叙事纪律的说明。 上述贡献 2 和 3 刻意不作为一串独立技巧来呈现；每一项都是服务于”先产出、再巩固一个未被污染信号”这一单一主旨的手段。在消融实验（第 6 章）中，每个子组件都必须对着这一主旨挣到自己的位置；任何在干净信号机制下不改善遗忘的子组件，将作为负面结果如实报告，而非强行辩护。

1.4 论文组织结构

本文余下部分组织如下。第 2 章综述持续学习、经验回放、智能体强化学习等相关工作并定位本文的位置。第 3 章给出问题定义，形式化 GRPO 优势估计的两类信号污染并重新表述为两阶段问题。第 4 章是核心方法章，分别介绍持续学习目标、七桶回放缓冲区、U 形块重加权、沙箱 winner 同步会话、三智能体在线多轮构造。第 5 章给出基于 verl 的系统实现。第 6 章给出评测基准、消融实验设计与分析维度。第 7 章总结全文并展望未来工作。

关于实验状态的说明。 截至本文写作时，训练系统代码已全部完成并通过约 290 项单元测试，但受限于 64 卡 GPU 集群与真实模型权重的可用性，21 个消融实

验尚未实际运行。因此第 6 章呈现的是**实验设计与评测协议**，所有依赖训练结果的数字以 **[TODO: 训练后回填]** 标记，本文不编造任何实验结果。

第 2 章 相关工作

本章综述与本文密切相关的工作，并定位本文的切入点。相关工作分为四组：经验回放与持续强化学习（A 组）、大语言模型智能体的持续学习与强化学习训练（B 组）、图形界面智能体的强化学习与持续学习（C 组）、以及经典持续学习基础方法（D 组）。

2.1 经验回放与持续强化学习

经验回放是缓解灾难性遗忘的主流手段。Zhang 等人^[8]提出回放增强的持续强化学习，通过存储全部历史经验并混合在线/离线学习与行为克隆，在无需任务标识的情况下显著减少遗忘。需要指出的是，“存储全部历史经验”是 A1 的设定（假设缓冲区容量无界）；在真实大规模智能体场景下这不可行，本文采用的是有限容量（25k）+ 桶内淘汰的缓冲区（第 4 章 §4.2）。Rolnick 等人提出的 CLEAR^[4]在任务无关设定下证明，朴素的经验回放配合在线/离线混合学习（V-trace 与行为克隆）即可大幅缓解遗忘，追平需要任务标识的 EWC、P&C 等方法；并指出缓冲区受限时随机丢弃即可接近无限缓冲区的效果。CLEAR 是本文消融实验中的基线方法（记为 R0）。

在 CLEAR 基础上有诸多改进。Pathmanathan 等人^[6]用好奇心同时检测任务边界与作为缓冲区保留优先级，提出按任务分区的保留策略，替代先进先出或蓄水池采样。Wang 等人^[5]从正则化强化学习目标推导出最优回放分布，表明基于 TD 误差的占据比率可将离策略数据拉向在线策略的最优分布，是本文优先级加权设计的理论基础。Zheng 等人^[7]用保持奖励分布的核心集实现 10 倍缓冲区压缩而性能不显著下降。此外，基于扩散模型的轨迹回放^[9]用生成分布替代固定样本回放，双

缓冲区设计^[10] 用短期与长期缓冲区保持任务多样性，蓄水池采样的改进^[11] 揭示了其对新数据的隐式偏置，自适应缓冲区^[12] 指出固定数据混合比在离线到在线迁移中存在局限。

上述方法共享一个隐含假设：被回放的经验携带着忠实的学习信号。本文指出，在多轮智能体强化学习中这一假设会在源头破裂，且无法用任何回放或正则化修复——这是本文区别于上述工作的核心切入点。

2.2 大语言模型智能体的持续学习与强化学习训练

大语言模型场景下的持续学习与强化学习训练是近年热点。Lai 等人^[2] 发现强化微调天然缓解持续后训练中的遗忘，性能接近多任务训练，而监督微调导致严重遗忘——这为本文采用强化学习而非监督微调做持续更新提供了动机。Wang 等人^[1] 直接指出，把 PPO/GRPO 搬到多轮智能体场景会因 Echo Trap（重复记忆路径导致多样性坍塌）而训练崩溃，本文的熵正则项即针对此失效模式。Surana 等人的综述^[13] 提出 LLM 强化学习 rollout 策略的 GFCR 分类法，其中 Replay 阶段涵盖经验回放与自进化课程，与本文的回放缓冲区及自适应课程相关。

在持续学习机制上，Zhang 等人的 RetroAgent^[14] 用 SimUtil-UCB 检索策略平衡相关性、历史效用与探索；Wang 等人的 BEPA^[15] 每任务维护动态缓存轨迹，用在线成功覆盖旧轨迹以降低分布偏移；Li 等人^[16] 将持续强化学习解耦为分布式单任务强化学习加策略蒸馏。Wu 等人^[17] 证明 GRPO 的方差缩减由 mini-batch rollout 数量 $B = Q \times G$ （提示数乘组大小）驱动，而非组大小单独决定，这为本文 rollout 批量规模设定提供了依据。

2.3 图形界面智能体的强化学习与持续学习

图形界面（GUI）智能体是与本文最接近的智能体持续学习场景。Yao 等人^[3] 发现 GRPO 的马太效应使高概率正优势动作获得最大更新，熵衰减反而保留已学策略结构，因而天然抗遗忘（SFT 遗忘指标 -5.73 vs GRPO -0.62 ）。Liu 等人^[18] 指出 GUI 分布随时间变化，现有方法无法维持稳定的 grounding。Hu 等人^[19] 论证监

督微调无法处理长程信用分配、分布偏移与安全探索，强化学习对 GUI 自动化至关重要。

本文与上述 GUI 智能体工作的关键区别在于：本文聚焦**多轮、工具使用、办公/运维/财务等领域任务**的纯文本智能体，并显式处理多轮场景特有的信号污染（组内环境发散与前提漂移），而既有 GUI 智能体持续学习多在单轮或弱多轮设定下研究分布偏移。

2.4 经典持续学习基础方法

经典持续学习方法为本文提供了概念基础。Lopez-Paz 与 Ranzato 提出的梯度情景记忆（GEM）^[20] 在情景记忆上使用梯度不等式约束（而非蒸馏或等式约束），可实现正向与向后迁移。本文的回放缓冲区与 GEM 同属基于记忆的方法族，但不依赖任务标识，且将”保留什么”的判据从梯度约束改为抗遗忘价值优先级。

2.5 本文与已有工作的关系

综合而言，强化微调天然比监督微调抗遗忘^[2,3]，但**仍然会遗忘**；CLEAR^[4] 证明任务无关下朴素经验回放即可大幅缓解遗忘，是本文的基线。本文在其上叠加**能力分桶 + 抗遗忘优先级 + 块级重加权**，并把整套方法放进**真实多轮智能体**场景（含工具、沙箱、用户模拟），显式处理多轮信号污染——这是与既有持续强化学习文献（多在玩具或 GUI 环境）的关键区别。

第 3 章 问题定义

本章形式化本文研究的问题。我们首先给出基本设定与 GRPO 的优势估计，然后指出在多轮智能体场景下该估计的两类信号污染，最后把问题重新表述为两阶段形式并给出持续学习的目标度量。

3.1 基本设定

策略与算法。基座模型为 Qwen3.6-27B，采用 GRPO（无 critic）作为强化学习算法。对每个查询，策略 rollout 出 $M = 8$ 条轨迹，组内归一化得到每条样本的优势：

$$A_i = \frac{r_i - \text{mean}(r)}{\text{std}(r) + \epsilon}, \quad (3.1)$$

其中 r_i 为第 i 条轨迹的奖励， $\text{mean}(r)$ 、 $\text{std}(r)$ 为组内均值与标准差， ϵ 为数值稳定小常数。

数据与任务。评测与训练覆盖纯文本 195 个任务，跨 7 个能力域（详见第 6 章）。任务以多轮会话形式组织：每个会话含多条顺序查询，共享上下文。每个任务含一份声明（首条真实查询 q_1 、隐藏目标、判分准则、用户画像）与一份初始文件系统（沙箱种子）。

目标。持续学习的目标是最大化综合指标

$$\text{CL Score} = \text{New Task Perf} - \alpha \cdot \text{Old Task Forgetting}, \quad (3.2)$$

其中 $\alpha = 1.0$ ，New Task Perf 衡量新任务学习效果，Old Task Forgetting 衡量旧任务评测分数相对上一阶段的下降量。

3.2 GRPO 优势估计的隐含前提

贯穿全文的一个隐含前提是：式 (3.1) 的组内归一化优势估计**仅当** M 条轨迹的差异纯粹来自策略采样的随机性时才无偏。换言之，组内方差应当只反映“同一策略在同一查询下的采样随机性”，而不应混入环境噪声或数据前提错位。任何对这一前提的违反都会把与策略无关的方差注入 A_i ，从源头污染学习信号。第 4 章的 §4.4–4.5 即从源头守住这一前提。

3.3 多轮智能体场景的两类信号污染

在多轮智能体强化学习中，上述前提并不自动成立。本文识别出两类在源头破坏优势估计的污染。

3.3.1 组内环境发散

跨多查询会话，同一组的 M 个执行环境会逐渐分叉。设第 k 条查询处理后，第 j 个槽位的环境状态为 $e_k^{(j)}$ 。若不在查询边界对齐各槽位，则第 $k+1$ 条查询的 M 个槽位将从不同状态 $\{e_k^{(1)}, \dots, e_k^{(M)}\}$ 出发，组内比较就混入了历史路径差异：某个槽位可能并非因策略更差、而是因第 k 条查询留下的不利状态被惩罚。于是优势 A_i 吸收了随查询序号 k 累积的环境方差，且组内标准差 $\text{std}(r)$ 会被这一环境噪声放大，导致信用分配（credit assignment）方向出错。

3.3.2 多轮前提漂移

后续查询 q_{k+1} 通常依赖上一轮执行后的状态 e_k （例如“这份 PPT 第 3 页数据错了”）。记 $\phi(q_{k+1})$ 为 q_{k+1} 的前提函数，当状态 e_k 满足该前提时 $\phi(q_{k+1})(e_k)$ 为真。若 q_{k+1} 在数据采集时被静态写死，而训练时 e_k 是策略 π_θ 的随机输出，则

$$\Pr [\phi(q_{k+1})(e_k)] < 1, \quad (3.3)$$

且该概率随策略变强而**持续漂移**。前提失败会产生污染信号：模型要么因对不存在的问题硬编一个修复而被奖励，要么因如实指出该问题不存在而被惩罚；同时，前

提不成立的样本会把环境噪声注入组内归一化，稀释信号。

关键澄清。即便回流数据里含有真实的后续查询 $q_2, \dots, q_K$ ，它们同样是在采集时针对原始会话的执行结果写下的；训练 rollout 中策略对 q_1 产出的状态与该原始结果不同，这些真实后续查询的前提同样不成立。直接复用它们会重新引入前提漂移——这正是本文只保留种子 q_1 、在线构造后续轮次的原因。

3.4 两阶段重新表述

被污染的优势在下游无法修复：任何回放加权或 KL 锚定都救不回一个符号本就错了的梯度。这把持续智能体强化学习重新表述为一个**两阶段**问题：

1. **产出干净信号**（第 4 章 §4.4–4.5）：用沙箱 winner 同步会话调度消除组内环境发散 (§3.3.1)，用三智能体在线构造多轮查询消除前提漂移 (§3.3.2)，使式 (3.1) 的优势估计恢复无偏。
2. **巩固干净信号**（第 4 章 §4.1–4.3）：在干净信号之上施加持续学习目标、抗遗忘回放缓冲区与块级重加权，在不遗忘的前提下巩固学习成果。

本文的整个系统围绕这一两阶段主旨组织：没有任何一个组件被当作独立技巧来呈现，每一项都是服务于“先产出、再巩固一个未被污染信号”这一主旨的手段。

第 4 章 方法

本章是本文的核心方法章。我们研究**智能体大语言模型的持续强化学习**：线上策略 π_θ 周期性地用新采集的交互数据更新，需要在习得新行为的同时**不遗忘**已掌握的能力。本章方法围绕一个把两半工作焊在一起的核心观察来组织。

GRPO 通过对同一查询的 M 条 rollout 在组内归一化来估计每条样本的优势（式 (3.1)）。这个估计**仅当** M 条轨迹的差异纯粹来自策略采样的随机性时才是无偏的。任何其他方差来源——组内各槽执行环境不一致，或某条多轮查询的前提在实际状态中并不成立——都会把环境噪声注入优势，从源头污染学习信号。

因此本文把问题拆成两个互相咬合的部分。**其一，把信号做干净** (§4.4–4.5)：沙箱 rollout 调度器保证每组的起点位级一致，并在每个轮次边界把会话对齐到优胜轨迹；模拟用户流水线在线构造多轮后续查询，使每条后续查询的前提由构造保证成立。**其二，把干净信号用好且不遗忘** (§4.1–4.3)：一个四项持续学习目标把经验回放与策略正则注入 GRPO；一个按能力分桶的回放缓冲区按抗遗忘价值给轨迹排优先级；一个 U 形块级重加权方案把回放的信用分配到最关键的 token 上。图 4.1 给出系统总览。

本章先讲第二半部——如何巩固信号 (§4.1–4.3)，因为它界定了训练对信号的要求；明确了“需要什么样的信号”之后，再回到第一半部——如何在源头把信号做干净 (§4.4–4.5)。

图待绘制：图 1：系统总览。左半为干净信号产出（16×8 沙箱 rollout + 每轮 winner 同步 + 三智能体多轮构造循环），右半为持续学习 trainer（ L_{cl} 目标 + 7 桶优先级 buffer），中间虚线箭头标注「干净 advantage」连接两半

图 4.1 系统总览

4.1 持续学习目标

本文优化一个在标准强化学习损失上叠加回放与正则的四项目标：

$$L_{cl} = \lambda_1 L_{rl} + \lambda_2 L_{kl} + \lambda_3 L_{replay} + \lambda_4 L_{ent}. \quad (4.1)$$

各项含义与关键决策如下。 L_{rl} 是新鲜 rollout 轨迹上的 GRPO 损失，是主优化目标，全程 $\lambda_1 = 1$ 。 L_{kl} 通过对参考策略的逆向 KL 散度约束策略漂移， $L_{kl} = D_{KL}(\pi_{new} \parallel \pi_{ref})$ ； π_{ref} 应取初始策略 π_0 还是上一阶段 checkpoint π_{t-1} 是一个经验问题，由消融实验（第 6 章）回答。 L_{replay} 是回放缓冲区采样轨迹上的行为克隆项，

$$L_{replay} = \mathbb{E}_{(s,a) \sim \mathcal{B}} [-\log \pi_{new}(a \mid s) \cdot w], \quad (4.2)$$

其中每 token 权重 w 在 §4.3 定义。 L_{ent} 是在线 rollout 状态上的熵奖励项，本文在**所有配置中默认开启**（ $\lambda_4 = 0.001$ ）。 L_{reg} （参数 L2 正则）弃用，权重为 0，槽位让给 L_{ent} 。

两个设计选择值得明确指出。**第一，弃用参数空间正则。**一种常见的抗遗忘项惩罚权重移动 $L_{reg} = \|\theta - \theta_{prev}\|^2$ 。本文将其权重置零：参数距离是功能距离的劣质

代理，而 L_{kl} 直接在输出分布空间约束策略，既更精确又已足够，腾出的系数槽位让给 L_{ent} 。第二，在本设定下熵正则并非可选项。多轮智能体强化学习易陷入 Echo Trap^[1]——策略多样性坍缩，组内 M 条轨迹趋于一致，导致优势趋零、训练停滞。 L_{rl} 与 L_{replay} 都是 mode-seeking 的、会加速熵衰减，而 L_{kl} 只让 π_{new} 在形状上接近 π_{ref} ，并不阻止熵坍缩。因此本文一律开启 L_{ent} ，包括无持续学习手段的基线；在基线中关掉它会导致基线崩溃，从而把”纯强化学习下的遗忘”与”崩溃后的退化”混为一谈，破坏各配置间的可比性。

该目标通过 verl/GRPO trainer 唯一支持的钩子 (`actor.set_loss_fn`) 注入，**不 fork 框架**：回放行以专用 mask 拼接进训练 batch，只贡献 L_{replay} ，绝不进入 PPO 分母。

4.2 按能力分桶的优先级回放缓冲区

回放缓冲区是 L_{replay} 取数的记忆体。两条原则使它区别于 CLEAR^[4] 这类朴素经验回放缓冲区。

按能力分桶，不按难度。 本文把缓冲区组织为 $B = 7$ 个对齐能力域的桶：Workflow (54 任务)、SysOps (52)、Dialogue (38)、Finance (18)、Communication (12)、Knowledge/Analysis (11)、OfficeQA (10)。难度是不稳定的划分轴——随策略变强，任务难度会漂移、迫使不断重新分类——而能力是稳定的。OfficeQA 单独成桶，避免并入 Knowledge 被稀释；不纳入多模态任务，因模态不同、数据太少、目标不一致。每个桶按一个平方根、带保底的配额分配获得额度，

$$q_i = q_{min} + (C - B q_{min}) \cdot \frac{n_i^{0.5}}{\sum_j n_j^{0.5}}, \quad (4.3)$$

其中 n_i 是桶 i 的任务数， C 是总容量 (25k)。平方根指数让大桶获得更多空间但不按比例膨胀，硬保底 q_{min} 防止小而独特的能力 (如 OfficeQA) 被挤空。关键在于，**淘汰是桶内的**：已满的桶只淘汰自己桶内优先级最低的轨迹，新任务永远无法淘汰其他桶的轨迹。这正是让”按能力分桶”主张可落地的机制——不同能力无法互相侵占。表 4.1 给出 25k 容量下的配额示例。

表 4.1 25k 总容量下各桶平方根配额分配示例 ($q_{min} = 2000$)

桶	任务数 n_i	平方根占比	配额 q_i
Workflow	54	0.174	≈ 4319
SysOps	52	0.172	≈ 4272
Dialogue	38	0.147	≈ 3938
Finance	18	0.101	≈ 3337
Communication	12	0.083	≈ 3092
Knowledge/Analysis	11	0.079	≈ 3047
OfficeQA	10	0.076	≈ 2995
合计	195	1.000	25000

按抗遗忘价值排序，不按奖励。桶内轨迹按一个融合四个抗遗忘信号的优先级保留，

$$\text{priority}_i = f(\text{forgetting_risk}_i, \text{rarity}_i, \text{diversity}_i, \text{difficulty}_i), \quad (4.4)$$

其中遗忘风险（当前策略在该轨迹上回退了多少）权重最高。本文刻意避免按奖励绝对值排序：随训练推进奖励整体上升，按奖励排序的缓冲区会系统性淘汰较旧轨迹、退化为滑动窗口，从而违背持续回放的初衷。采样为两级——先采桶（配额比例与均匀混合，对长期空闲的桶给饥饿提升），再在桶内按优先级加权随机采样（而非 top- k ），使回放不会反复强化少数“明星”轨迹。

4.3 回放的 U 形块重加权

§4.2 的优先级决定了哪条轨迹值得回放，但一条多轮轨迹内部的 token 并非同等重要——早期的分支决策与最终的结论段，比中间的执行细节更值得巩固。本文因此把信用进一步细分到 token 级： L_{replay} 中的权重 w 由 trajectory 级项与 token 级项相乘合成。具体地，对轨迹 i 的 token t ,

$$w_t^{(i)} = \text{normalize}\left(\text{clip}\left(\text{priority}_i \cdot \frac{\gamma^{\text{block}(t) + \delta^{K_i - 1 - \text{block}(t)}}}{2}, q_5, q_{95}\right)\right), \quad (4.5)$$

其中 priority_i 是 §4.2 的轨迹优先级，第二个因子是仅作用于 response token 的 **U 形块权重**。块索引 $\text{block}(t) \in \{0, 1, \dots, K_i - 1\}$ 从 0 起始， K_i 因轨迹而异。

动作块切分。本文不对所有 token 等同处理、也不按原始 token 位置衰减，而是按结构标签 (<think>、<toolcall>、<observation>、<final_answer>) 把每条轨迹切成 K_i 个动作块。 K_i 因轨迹而异；块内 token 共享权重。当轨迹无法解析成块（或只解析出单块）时，回退到等长切分。这让权重曲线对齐 agent loop 的语义边界，而非任意 token 偏移。

为什么是 U 形。块权重对首块（早期、高方差的分支决策——选哪个工具、走哪条路）和末几块（结论生成段）同时赋高权，对中间的执行细节降权。该方案的早期版本用单调衰减加仅在 final-answer 段做单点提升；本文改为对称 U 形，因为轨迹末端的重要性并不局限于 final-answer 这一个 token 段——靠近末端的多个块（结论前的总结、关键判断）同样重要，单点提升无法覆盖一整个区域。当 $\gamma = \delta$ 时方案连续可调：令 $\gamma = \delta = 1$ 使每个块权重恒为 1，恢复为内置的均权基线，于是均权与 U 形两种配置之间唯一的变量就是 (γ, δ) 的取值。最后，本文在归一化前对 **乘积**（而非仅优先级）做 [5%, 95%] 分位裁剪，因为极端权重来自优先级与块位置的相乘放大，只有裁剪最终乘积才能控制住。图 4.2 展示了不同 (γ, δ) 下的 U 形曲线。

图待绘制：图 3：U 形块权重曲线。权重对归一化块位置 $\text{block}(t)/K_i$ ， $\gamma = \delta \in \{1.0, 0.92, 0.88\}$ ，展示 1.0 时的水平线（均权）与 γ 减小时 U 形渐显

图 4.2 U 形块权重

4.4 沙箱 Rollout 与 Winner 同步会话

以上三节 (§4.1–4.3) 都默认喂进来的学习信号是干净的——优势只反映策略差异、多轮查询的前提都成立。但正如本章开头所述，这个前提在智能体多轮场景下并不自动成立，需要主动保证。本节与下一节就回到第一半部：在源头把信号做干净。本节先处理组内环境一致性。

每个智能体任务在云沙箱中执行：策略推理（27B 前向，同时产出训练所需的逐 token 对数概率）在沙箱外的 GPU 集群上运行；动作（装包、写文件、跑命令）在沙箱内运行——那里状态可进程级快照、可数百实例并发隔离。轨迹直接从 rollout 引擎的**原生**输出（token id、response mask、行为对数概率）收集，而非由外部代理事后重建。

一个 rollout step 并行跑 16 个会话，每会话一组 $M = 8$ 个槽，共 128 个并发实例。会话内，每条查询的处理如下：8 个槽从同一母版派生，因此起点**位级一致**；策略随后 rollout 出 8 条轨迹，允许它们在执行中发散（这种发散正是优势方差的来源）；选出优胜者；并在下一条查询前把全部 8 个槽的系统状态与对话历史都对齐到优胜者。会话结束时整体销毁，不回写母版。

这种每轮 winner 同步正是在多查询会话中保持 GRPO 优势干净的关键。若不同步，下一条查询的 8 个槽将从不同状态出发，组内比较就会混入历史路径差异；某个槽可能并非因策略更差、而是因上一条查询留下的不利状态被惩罚，破坏信用分配，且组内标准差会吸收随查询序号累积的环境方差。本文**只在查询边界**同步；查询执行过程中的发散被保留。一个实现约束是：会话内 winner 实例必须存活——它是累积的进程态与内存态的唯一活载体——所以只淘汰七个输家、由存活的 winner 派生出替补。图 4.3 给出单会话时序。

正史纠正机制。 winner 同步包含两个必须同时执行的纠正：系统状态对齐与**对话正史**（会话历史 H_t ）对齐。本文显式区分这两条纠正，因为它们消除的是两类不同的非策略方差来源。系统状态对齐消除环境分歧——8 个槽在执行同一查询时各自修改了不同的文件、安装了不同的包，若不对齐，下一条查询的起点环境就不一致。对话正史对齐消除上下文分歧——8 条轨迹各自产生了不同的对话消息（不同的工具调用序列、不同的中间回答），若不对齐，下一条查询的组内比较就会把”上一轮说了不同的话”这一历史路径差异混入优势估计。

具体地，设第 k 轮 8 条轨迹的对话历史分别为 $h_k^{(1)}, \dots, h_k^{(M)}$ ，优胜者为 τ^w ，其历史为 $h_k^{(w)}$ 。正史纠正将全部槽的历史统一覆写为优胜者历史：

$$\forall j \in \{1, \dots, M\}, \quad h_k^{(j)} \leftarrow h_k^{(w)}, \quad H_{k+1} = H_k \cup \{h_k^{(w)}\}, \quad (4.6)$$

其中 H_k 是截至第 k 轮的会话正史（历代 winner 消息的累积拼接）。纠正后，第 $k+1$ 轮的 8 个槽从统一正史 H_{k+1} 出发接收下一条查询，使组内方差仅反映本轮策略采样的随机性，而非上一轮的历史路径差异。

正史纠正与系统状态对齐**缺一不可**。仅对齐磁盘而不纠正正史：下一条查询的 8 个槽虽在同一文件系统上运行，但各自看到不同的对话上下文，出题 agent 或策略本身可能因上下文差异产生不同行为，组内比较仍混入历史路径噪声。仅纠正正史而不对齐磁盘：8 个槽虽看到相同对话历史，但底层环境状态不同（文件内容、安装的包不同），导致相同输入产生不同输出，组内比较仍混入环境噪声。只有两者同时纠正，才能使下一条查询的组内方差纯粹来自策略采样。

正史 H_t 同时服务两个下游消费者，保证了”用来出下一题的上下文”与”训练时策略看到的上下文”完全一致。出题 agent 基于优胜者正史 H_t 生成下一条查询 $q_{t+1} \sim Q(\cdot \mid p, R_t, H_t)$ ，使后续查询的前提建立在优胜者实际达成的状态之上；训

练时策略的输入也是同一份 H_t 作为 prompt 前缀，使 rollout 与训练的前向一致。这与系统状态对齐的语义对称：磁盘对齐保证环境在 rollout 与训练间一致，正史纠正保证对话上下文在 rollout 与训练间一致。

图待绘制：图 4：Winner 同步会话时序。单会话从母版派生 8 个位级一致槽位；对每条查询，8 路并行 rollout → 选 winner → 全部槽位（磁盘 + 历史）同步到 winner → 下一条 query；会话结束销毁

图 4.3 Winner 同步会话

实现注记：在标准框架内落地 winner 同步。上述会话调度无法用标准强化学习框架（如 verl/GRPO）的默认 rollout 表达。默认范式是”把整批提示一次性生成完、再统一打分”——而 winner 同步是查询之间的耦合（下一条查询的起点取决于上一条的优胜者），单次批量生成内部无法表达这种顺序依赖。本文因此把 rollout 阶段的控制权收归己有：**由本文编排会话循环（16×8 槽 + 每查询后 winner 同步），但每一步的单条生成仍调用框架原生的生成引擎**，使 token 与对数概率由框架直接产出，而非由外部代理事后重建（这是保证 L_{rl} 的重要性比率与训练前向数值一致的前提）。具体地，本文利用框架预留的 rollout 管理器注入点，以一个自定义的 rollout 管理器替换默认实现：它对外仍满足”输入一批提示、输出一批带 response mask 与对数概率的轨迹”的契约（从而训练循环的其余部分——优势计算、损失、参数更新——完全不变），对内则把这批提示当作种子查询交给本文的会话调度器，跑完 16×8 加 winner 同步后，再把收集到的轨迹组装回框架要求的批张量格式。**这一替换是非侵入的**：不 fork 框架、不改其训练循环，只在官方注入点挂入；与本文

注入持续学习损失用的是同一种“零源码改动”策略 (§4.1)。其代价是一个适配层——把可变长度的多轮轨迹组装成框架的稠密批张量。

4.5 经模拟用户在线构造多轮查询

信号污染的第二个来源是多轮数据。一条后续查询通常引用上一轮的结果（例如“那份 PPT 第 3 页的数据错了”）。当这样的后续查询在数据采集时被写死、而实际状态 e_k 是策略的随机函数时，该后续查询的前提仅以概率 $\Pr[\phi(q_{k+1})(e_k)] < 1$ 成立，且该概率随策略变强而漂移。前提失败会产生污染信号：模型要么因对不存在的问题硬编一个修复而被奖励，要么因如实指出该问题不存在而被惩罚。

本文通过**在观察到实际状态之后再在线构造后续查询**来消除前提漂移。从每个真实会话中只保留第一条查询作为种子（保住真实的用户意图分布），并在 winner 同步边界用三个协作智能体生成后续轮次。值得强调的是，回流数据里本就含有真实的后续查询，但它们同样是在采集时针对原始会话的执行结果写下的；训练 rollout 中本文策略对首条查询产出的状态与该原始结果不同，这些真实后续查询的前提因而同样不成立——直接复用它们会重新引入前提漂移。**这正是本文丢弃真实后续查询、只保留种子首条查询的原因**：真实数据的价值在其首条查询所携带的真实意图分布，而非其与特定一次执行绑定的后续轮次。三个协作智能体是：

- 一个**观察智能体**（无人设），对比该轮**前后的** winner 沙箱状态，从由此得到的环境**差异**（本轮新增/修改/删除的文件及其内容，加系统状态变化，只读）中收集一份中立、结构化的报告 $R_t = \text{Obs}(e_t^w)$ ，尤其捕获中间结果与最终结果。关键在于观察智能体**只基于真实状态**：它**根本不接收 actor 的轨迹或声称**。这使反奖励作弊成为**结构性的**——actor 的叙事从不进入观察判断，完成度只能反映被核实的真实效果，策略无法靠“声称完成”骗分。观察智能体同样独立于策略模型（不同模型/后端），观察与动作解耦。
- 一个**出题智能体**，携带 42 个固定人设之一（职业、偏好、用户画像，以及一个观察偏好：整体 vs 细节、形式 vs 内容）。人设**在每个会话开始时随机抽取一个，并在整个会话内保持不变**；它生成下一条查询 $q_{t+1} \sim Q(\cdot \mid p, R_t, H_t)$ ，其中 H_t 是 winner 衍生的会话历史。

- 一个**奖励模型**，从同一份报告按两条通道对该轮打分：观察智能体的状态差异（完成度的真值）加在 R_t 上透传携带的 actor 轨迹（判安全/鲁棒性）， $r_t = \text{Reward}(R_t, \text{rubric})$ 。本轮无任何变化（空差异）则短路为 0、不调 judge。

报告 R_t 同时服务两个下游消费者，这保证了”用来打分的事实”与”用来出下一题的事实”完全一致，并把客观证据收集（观察智能体）与主观立场（出题智能体）解耦。由于后续查询是在观察智能体检视实际状态之后才写出的，其前提由构造成立， $\Pr[\phi(q_{t+1})(e_t^w)] \approx 1$ 。把奖励的完成度落在观察到的实际效果（文件是否真的生成、内容是否正确）而非任何文字声明上，是反奖励作弊的关键——而由于观察智能体本身从不读轨迹，这种 grounding 是**结构性的**，不依赖 judge 的尽责程度。图 4.4 展示了三智能体在 winner 同步边界的协作环。

图待绘制：图 5：三智能体多轮构造。*winner-sync* 边界处：观察智能体读 *winner* 沙箱差异（仅状态）→ 产出报告 R_t （透传携带轨迹）； R_t 分发给奖励模型（状态差异 → 完成度，透传轨迹 → 安全/鲁棒性）与人设化出题智能体（下一查询或结束会话）

图 4.4 三智能体多轮构造

四维抗坍塌。自对话的已知失效模式是模式坍塌——后续查询趋同于少数模板腔，熵随轮数衰减。本文用四个正交维度对抗它：（1）人设多样性，引入 42 个多样人设，每会话随机抽一个，把同一份客观报告问出不同侧面；（2）轮数压制，后续查询轮数在 $\{1, 2, 3\}$ 中抽样，截断自回归生成链，避免模型学到”固定第 N 轮结束”的捷径；（3）状态逐轮演化，每轮 *winner* 状态都被上一条后续查询改变，同一个

人设每轮看到的报告也不同；(4) 出题智能体多模型轮换，出题智能体在 4 个跨厂商模型——anthropic/claude-sonnet-5、deepseek/deepseek-v4-pro、qwen/qwen3.7-max、moonshotai/kimi-k2.6——之间轮换，每 5 次提问后切换。前三个维度已有文献先例（人设多样性^[4]、轮数截断^[1]、输入漂移），第四个维度是本文新增的**模型层异质性**：人设决定”问什么”，模型决定”怎么问”，两者从不同维度注入多样性，联合效果大于任一单维度。

由人设耐心治理的失败兜底。本文不设固定的重试上限，而是把”是否重试”做成模拟用户的属性，由每个人设携带的两个量治理：初始耐心 $P_0(p)$ 与基础扣减 $d_0(p)$ 。设 $k = 1, 2, \dots$ 标记会话内连续失败的轮次。第 k 次失败时耐心按一个指数增长的扣减衰减，

$$P_k = P_{k-1} - d_0(p) 2^{k-1} = P_0(p) - d_0(p) (2^k - 1). \quad (4.7)$$

控制器随后以概率 $\text{clip}(P_k, 0, 1)$ 重做该轮，否则以 `<end_session>` 结束会话；特别地 $P_k < 0$ 时确定性终止。让 P_0 与 d_0 均由人设携带，解耦了两个不同的性格特质——初始容忍度与升级速度。由于扣减每次翻倍，耐心在 $O(\log_2(P_0/d_0))$ 次失败后越过零，因此重试天然有界、无需单独设上限。

一个有用的副作用是**自适应课程**：出题智能体始终针对当前策略的实际输出挑刺，因此策略越强、残留缺陷越细微，它提出的后续查询也越精细，自动跟随能力前沿，无需人工设计难度调度。

综上，在定义了如何产出干净信号 (§4.4–4.5) 以及如何在不忘的前提下巩固它 (§4.1–4.3) 之后，本文设计受控消融实验，逐一隔离每个组件的边际贡献（第 6 章）。三个智能体与奖励模型的完整提示词见附录 A。

算法 4.1 winner 同步会话 + 三智能体在线多轮构造（单会话）

输入： 母版镜像 M ，种子查询 q_1 ，人设 $p \sim \text{Persona}$

- 1: 从 M 派生 8 个位级一致槽位 $\{s_1, \dots, s_8\}$
 - 2: $H \leftarrow \emptyset$ ▷ 会话正史
 - 3: **for** $t = 1, 2, \dots$ 直到 $\langle \text{end_session} \rangle$ **do**
 - 4: 8 路并行 rollout: $\{\tau_j\}_{j=1}^8 \leftarrow \pi_\theta(\cdot \mid q_t, s_j)$
 - 5: 选 winner $\tau^w \leftarrow \arg \max_j r(\tau_j)$
 - 6: winner 同步: $\forall j, s_j \leftarrow \text{sync}(s_j, \tau^w)$ ▷ 系统状态 + 历史对齐
 - 7: $e_t^w \leftarrow \text{winner 沙箱状态}; R_t \leftarrow \text{Obs}(e_t^w)$ ▷ 观察智能体，仅状态
 - 8: $r_t \leftarrow \text{Reward}(R_t, \text{rubric})$ ▷ 奖励模型，空差异短路为 0
 - 9: $H \leftarrow H \cup \{\tau^w\}$
 - 10: $q_{t+1} \leftarrow Q(\cdot \mid p, R_t, H)$ ▷ 出题智能体，或 $\langle \text{end_session} \rangle$
 - 11: **end for**
 - 12: 销毁全部槽位，不回写母版
 - 13: **return** 收集的轨迹 $\{(\tau_t^w, r_t)\}$ 入桶
-

第 5 章 系统实现

本章给出持续学习训练系统的工程实现。系统以 verl 0.8.0 为训练框架，通过零源码改动的注入点接入持续学习损失与回放缓冲区；沙箱 rollout 与三智能体在线构造解耦为独立模块。本章重点描述几个关键的工程决策与解耦设计。

5.1 基于 verl 的零侵入集成

本文以 verl 0.8.0 (pip 安装, 不 fork) 作为强化学习训练框架。verl 自身没有经验回放缓冲区的概念, 本文的缓冲区作为纯 Python 模块外挂接入; 持续学习损失则通过框架暴露的注入点接入, 无需修改框架源码。

损失注入。verl 在 `actor.set_loss_fn()` 接口处支持自定义策略损失函数。本文的 `trainer/cl_loss.py::make_cl_loss` 返回一个 verl 兼容的损失闭包, 实现式 (4.1) 的四项目标。回放行通过专用 mask 拼接进训练 batch: 回放行的 PPO `response_mask` 置 0 (从而不进入 PPO 损失的分母), 另设 `replay_response_mask` 标记真实回放 span, 使 L_{replay} 独占该 mask。这一双 mask 设计保证了回放行只贡献 L_{replay} 、绝不污染 PPO 损失。

缓冲区钩子。本文通过 `install_buffer_hooks` 在训练器的参数更新阶段挂入缓冲区交互: 每个 step 从缓冲区采样回放轨迹、组装成回放行拼入 batch, 训练后再把当前 rollout 的新轨迹入库。该钩子 patch 训练器的 `_update_actor` 方法, 不使用全局 monkey-patch。

Fully Async 对接。为支持推理与训练分离部署 (§5.4), 本文实现了 `trainer/verl_async_ru` 将持续学习 mixin 注入 verl 的 Fully Async Policy 训练器底层类并重新注册为 Ray actor, 在 `_fit_update_actor` 处挂入缓冲区钩子。 L_{replay} 本身是 off-policy 的,

与 fully-async 的异步样本天然契合。

5.2 沙箱架构与厂商无关后端

动作在内、推理在外。每个智能体任务在云沙箱中执行。策略推理（27B 前向与采样，同时产出训练所需的逐 token 对数概率）在沙箱外的 GPU 集群上以 vLLM 运行；动作（装包、写文件、跑命令）在沙箱内运行，那里状态可进程级快照、可数百实例并发隔离。128 个沙箱不可能各带一份 27B 模型，而动作必须落到“那台用户机器”的系统状态上，因此推理与动作在物理上分离。轨迹直接从 rollout 引擎的原生字段（token id、response_mask、rollout_log_probs）收集，而非由外部代理事后重建，从而保证 L_{rl} 的重要性比率与训练前向数值一致。

16×8 winner 同步调度。每个 rollout step 并行 16 个会话，每会话 8 个槽位，共 128 个并发实例。会话内每条查询：8 槽从同一母版派生（位级一致）→ 8 路并行 rollout（中途发散即优势方差来源）→ 选 winner → 8 槽系统状态与对话正史都对齐到 winner → 下一条查询。会话结束销毁全部、不回写母版。会话内绝不淘汰 winner（它是累积进程态与内存态的唯一活载体），只淘汰 7 个输家、由存活 winner 派生替补。

厂商无关的沙箱后端。本文将沙箱后端设计为接口与实现解耦的注册表：SandboxClient 协议定义接口契约，register_backend 与 make_sandbox 按名选择后端。已实现 local（本地开发）与 e2b（腾讯云）两个真后端，aliyun（阿里云 AgentBay）留空 stub。换或加沙箱厂商时，rollout 循环零改动。

5.3 模型选型

各角色的模型选型如表 5.1 所示。选型的单一信源为 configs/agents.yaml，代码与启动脚本统一读取该文件。所有角色共用同一 sufy 网关端点（openai.sufy.com/v1）与同一密钥 SUFY_API_KEY，但通过三套独立的环境配置（OBSERVER_*/USERSIM_*/JUDGE_*）使观察、出题、判分互不为同一模型，抗 self-preference 偏置。判分模型冻结且能力不低于策略，作为一把不变的尺子衡量遗忘，避免版本漂移污染遗忘

度量。

表 5.1 各角色模型选型（单一信源：configs/agents.yaml）

角色	选型	温度	说明
Actor / 基座策略	Qwen3.6-27B	rollout 1.0	训练对象，本地权重
观察智能体 Observer	openai/gpt-5-mini	0.0	客观取证，确定性
出题智能体 Questioner	anthropic/claude-sonnet-5（主）	0.9	4 模型轮换，抗坍缩
奖励 / Judge	anthropic/claude-4.8-opus	0.0	冻结，与评测同构

出题智能体采用**多模型轮换**：在 4 个跨厂商模型——anthropic/claude-sonnet-5、deepseek/deepseek-v4-pro、qwen/qwen3.7-max、moonshotai/kimi-k2.6——之间轮换，每 5 次提问后切换到下一个 (§4.5)。轮换池的模型来自不同厂商，最大化输出风格异质性：人设决定”问什么”，模型决定”怎么问”，两者从不同维度注入多样性。

观察智能体的 LLM 调用是**可选的**：默认 use_llm=False，走确定性取证（沙箱 before/after 内容级差异为真值），零模型调用；仅在需要模型归纳或差异标记时才启用 LLM。这一设计避免了”让奖励模型去观察”的高消耗，同时反奖励作弊的取证层始终运行。

5.4 GPU 部署与训练精度

部署。64 张 H800，采用推理与训练分离的 40+24 配置：推理组 40 卡（5×TP8 vLLM），训练组 24 卡（FSDP2）。在智能体工具执行延迟 4–8 秒/轮的 Deep Research 场景下，分离部署比 Colocate 64 快约 3–7%，因训练与 rollout 可并行。Colocate 64 为后备方案。

异步框架。采用 verl 的 Fully Async Policy，通过 staleness_threshold=0.3 严格控制异步程度（最多 30% 样本来自旧策略），trigger_parameter_sync_step=4，require_batches=4，partial_rollout=False。该方案可平滑退化为同步（staleness_threshold=0），是更稳的工程选择。27B 单次权重同步约 4–5 秒（参考 verl 实测 30B 为 4.38 秒）。

精度。全栈 BF16 混合精度：训练参数、激活、梯度均 BF16；保留 FP32 主权重与 FP32 Adam 一阶/二阶动量以保证更新数值稳定。FP8 不进入主路径——27B + 智能体 + 持续学习三重叠加的 FP8 稳定性未验证，且 27B 显存余量充裕（训练侧约 50 GB/卡，80 GB 余 30 GB）。Phase 5 大 rollout 量下若吞吐成瓶颈，可考虑仅 rollout 切 FP8（训练保持 BF16），并配合 Token-level TIS 兜底。

5.5 工程解耦与可测试性

回放缓冲区与框架解耦。`replay_buffer/` 是纯 Python 模块，不 import verl，不依赖 Ray，可在 CPU 单机上独立单元测试。截至本文写作时，该模块含 100 余项单元测试。这一解耦使缓冲区的优先级、配额、淘汰、采样逻辑可在无 GPU 环境下充分验证。

三智能体离线 harness。配套 `scripts/agents_harness.py` 提供观察 + 出题 + 奖励三智能体的离线回路，无 GPU 即可跑通，利于复现与调试。

轨迹收集用框架原生字段。轨迹直接从 verl 的 `ToolAgentLoop` 原生输出收集，不写外部代理，保证 token 与对数概率由框架直接产出。

5.6 当前实现状态与残缺模块

截至本文写作时（2026-07），训练系统代码已全部完成，CPU 单机通过约 290 项单元测试（288 passed + 7 skipped，skip 均为“未装 verl/无 GPU”）。回放路径已在真实 H800 上通过 CUDA smoke（回放行反向、梯度大于零）。verl 集成的关键接线（损失闭包签名对齐、回放行双 mask 与跨长度填充、`forgetting_risk` 当前对数概率前向）均已审计修复并通过纯逻辑单测。

唯一阻塞正式训练的是外部依赖：64 卡 GPU 集群、真实 Qwen3.6-27B 权重、ClawEval 任务 manifest、沙箱 rollout 环境、用户 query 种子。残缺模块（Gap A-H）中，奖励模型、数据转换、沙箱调度、rollout 桥均已落地代码；全栈 1-2 step GPU smoke 待集群验证。21 个消融实验均未实际运行，checkpoint 产出为 0——这是第 6 章只能呈现实验设计而非结果的根本原因。

第 6 章 实验设计与分析

本章给出评测基准、消融实验设计与分析维度。**需要再次强调的是：**截至本文写作时，受限于 64 卡 GPU 集群与真实模型权重的可用性，21 个消融实验均未实际运行。因此本章呈现的是**实验设计与评测协议**，所有依赖训练结果的数字以 **[TODO: 训练后回填]** 标记，本文不编造任何实验结果。

6.1 评测基准

本文采用 ClawEval 作为评测基准。ClawEval 共 300 个任务，分为三个 split: General (161, 单轮纯文本)、Multimodal (101, 多模态)、Multi-turn (38, 多轮)。本文当前仅使用纯文本任务 (General 的 161 个，其中按能力域归入 7 桶共 195 个任务，含部分 Multi-turn 文本任务)。

评分公式。每个任务在三个维度上评分：完成度 $s_{completion}$ 、安全 s_{safety} 、鲁棒性 $s_{robustness}$ ，均为 $[0, 1]$ 浮点。综合分按

$$score = s_{safety} \times (0.8 \cdot s_{completion} + 0.2 \cdot s_{robustness}) \quad (6.1)$$

聚合，即安全为一票否决 ($s_{safety} = 0$ 则总分 0)，完成度为主、鲁棒性为辅。

Pass³ 标准。每个任务独立运行 3 次，三次全部通过才算通过 (Pass³)。这一严格标准降低了单次运行的随机性，更稳健地衡量真实能力。

奖励与评测同构。训练时的奖励模型与评测使用同一套三维判分准则与同一聚合公式 (式 (6.1))，由冻结的 judge 模型打分。这保证了”训练优化的目标”与”评测衡量的目标”一致，避免奖励与评测错位。

6.2 消融实验路线

本文设计 21 个受控消融实验，逐一隔离每个组件的边际贡献。每个对照只动一个变量，保证边际贡献可归因。实验路线如下：

1. **Phase 1 (B1): 纯强化学习遗忘基线。** $\lambda_1 = 1$, $\lambda_2 = \lambda_3 = 0$, 仅 $\lambda_4 = 0.001$ 熵正则。建立“无持续学习手段”下的遗忘下界。B1 必须开 λ_4 : 关闭熵正则会触发 Echo Trap 导致训练崩盘，测到的就不是真实遗忘量而是崩盘退化。
2. **Phase 2 (K1–K5, K2-R): KL 单独验证。** 扫描 KL 权重 $\lambda_2 \in \{0.01, 0.05, 0.10\}$ 与参考策略锚点 (π_0 初始 vs π_{t-1} 上一阶段)，并通过 K2-R 验证 KL 在有回放时是否冗余。选出 top-2 KL 配置供 Phase 4 组合。
3. **Phase 3 (R0-10k, R0-25k, R3, R4, R5, R4-w, R6, R4-K): 回放单独验证。** R0 为 CLEAR 基线（单缓冲区 + 蓄水池 + 均匀），分 10k/25k 两档容量以隔离“容量 vs 桶结构”；R3 加两级采样；R4 加抗遗忘优先级；R5 改用奖励优先级（与 R4 对照，验证“不用 reward 排序”的核心主张）；R4-w 加 U 形块权重；R6 提高回放权重；R4-K 加 KL 交互验证。
4. **Phase 4 (C1–C4): KL × 回放组合验证。** 用 Phase 2/3 的 top-2 配置做 2×2 网格，验证 KL 与回放是互补还是冗余，找最优组合。
5. **Phase 5 (S1, S2): rollout 规模扩展。** 在 Phase 4 最优配置下，从 1024×8 扩展到 4096×8，验证规模效应。

Phase 3 的关键对照轴如表 6.1 所示。

多轮查询构造方案 (§4.5) 不单列消融——实验已饱和，靠过程监控指标与 ClawEval Multi-turn 终点指标验证其效果。

表 6.1 Phase 3 关键对照轴（每行只动一个变量）

对照轴	实验组	验证目标
桶结构 vs CLEAR	R0-25k $\rightarrow$ R3	桶结构在长尾分布下是否补偿稀释
容量	R0-10k $\rightarrow$ R0-25k	单纯加大容量能否替代桶结构
是否用优先级	R3 $\rightarrow$ R4	抗遗忘优先级价值
优先级类型	R4 $\rightarrow$ R5	抗遗忘 vs 奖励优先级（核心主张）
优先级用法	R4 $\rightarrow$ R4-w	U 形块重加权综合效果
λ_3 权重	R4 $\rightarrow$ R6	回放权重对新/旧任务平衡
KL $\times$ 回放交互	R4 $\rightarrow$ R4-K	回放在有 KL 时是否冗余

6.3 过程指标与早期预警

除终点指标（CL Score、New Task Perf、Old Task Forgetting）外，本文监控一系列过程指标，用于早期预警与归因诊断：

- **输出熵**： $H(\pi_{new}(\cdot | s))$ 在在线 rollout 状态上的均值曲线。Echo Trap 早期预警——前 100 step 下降超 50% 即需调大 λ_4 。
- **轨迹多样性**：单查询内 8 条轨迹的 distinct-n / self-BLEU。直接量化 Echo Trap——组内轨迹趋同则优势退化、该查询失效。
- **KL 趋势**：各 step 的 $D_{KL}(\pi_{new} || \pi_{ref})$ 曲线，监控策略漂移是否受控。
- **回放/在线损失比**： L_{replay}/L_{rl} 的变化趋势，判断回放与在线学习的平衡性。
- **优势分布**：新旧任务 rollout 的优势均值与方差，诊断梯度信号是否稳定。
- **梯度范数**：各损失分量的梯度 L2 范数，检测某一分量是否主导训练。
- **缓冲区各桶动态**：各桶的 size、填充率、淘汰数、信号权重，由 `trainer/replay_metrics.py` 落 sidecar 日志。

这些过程指标在训练全程实时采集，既是调参依据，也是论文中分析”某个组件为何有效/无效”的证据。

6.4 预期结果与待回填项

正式训练完成后，预期回填以下结果：

- **头条数字**：相对 CLEAR (R0) 与纯强化学习基线 (B1) 的遗忘量与 CL Score 改进幅度 [TODO: 训练后回填]。
- **逐组件消融表**：Phase 2/3 各对照轴的 New Task Perf、Old Task Forgetting、CL Score [TODO: 训练后回填]。
- **U 形权重有效性**：R4 vs R4-w 的对比，验证块级重加权是否改善遗忘 [TODO: 训练后回填]。
- **优先级类型对照**：R4 (抗遗忘优先级) vs R5 (奖励优先级)，验证”不用 reward 排序”的核心主张 [TODO: 训练后回填]。
- **过程曲线**：输出熵、轨迹多样性、KL 趋势随 step 的变化 [TODO: 训练后回填]。

依据本文的叙事纪律，任何在干净信号机制下不改善遗忘的子组件，将作为负面结果如实报告，而非强行辩护。

第 7 章 总结与展望

7.1 工作总结

本文研究智能体大语言模型的持续强化学习，针对”在习得新行为的同时不遗忘已掌握能力”这一核心难题，提出了一个围绕单一主旨组织的端到端方案：**在持续多轮智能体强化学习中，防止灾难性遗忘不仅要巩固学习信号，更要先保证信号在源头上没有被污染**。全文每一部分都只是两件事之一——在源头把信号做干净，或在不遗忘的前提下巩固这个干净信号。

本文的主要贡献如下：

1. **对持续智能体强化学习的源头级重新表述**。本文识别并形式化了 GRPO 学习信号的两类污染——组内环境发散与多轮前提漂移——它们发生在任何巩固之前，且无法用回放或正则化修复。这一重新表述把持续智能体强化学习从单纯的”抗遗忘”问题扩展为”先产出干净信号、再巩固”的两阶段问题。
2. **一套产出干净信号的数据与 rollout 系统**。winner 同步会话调度在多查询会话中恢复组内位级一致起点，消除组内环境发散；观察、出题、奖励三个协作智能体在线构造前提成立的多轮数据，根除前提漂移，并兼作跟随策略能力前沿的自适应课程。其中观察智能体以沙箱 before/after 内容级差异为真值、不接收 actor 轨迹，使反奖励作弊成为结构性机制。
3. **一套与干净信号匹配的巩固方法**。以零源码改动注入 GRPO 的四项持续学习目标（强化学习、逆向 KL、经验回放、熵正则）；按能力而非难度分区、按抗遗忘价值而非奖励排序的七桶优先级回放缓冲区，桶内淘汰、禁止跨桶挤出；U 形块级回放重加权，把信用集中到每条轨迹的早期分支决策与结论段。

4. **系统实现与实验设计**。基于 verl 0.8.0 零侵入落地的端到端训练系统，代码已全部完成并通过约 290 项单元测试；以及在 27B 智能体、七个能力域、Pass³ 评测基准上的 21 个逐组件受控消融实验设计。

7.2 局限性

本文工作存在以下局限：

1. **U 形与优先级融合权重未实证调优**。 $\gamma = \delta = 0.88$ 、优先级四信号融合权重 $(0.4, 0.2, 0.2, 0.2)$ (v1 实跑因缺 embedding pipeline 暂禁用 diversity, 重归一化为 $(0.5, 0.25, 0, 0.25)$) 均为起步值，待 Phase 3 数据校准。短轨迹（小 K_i ）下 U 形可能近乎消失，需更小 γ 。
2. **模拟用户分布失真与幻觉**。42 个人设的分布未必等于真实后续查询分布；观察不全时前提漂移以小概率回归，靠首轮真实锚点与前提成立率审计缓解。
3. **观察 grounding 强度等于取证能力**。反奖励作弊的强度上限等于观察智能体能取到的证据强度。二进制格式提取的真值核对需库与真实文件，瞬态或被覆盖的中间产物需事件流（当前只看净变化）。
4. **winner 同步进程态保真**。平台若只支持磁盘快照，替补槽的进程/内存态可能与 winner 不一致，这是头号待验证点。
5. **实验尚未运行**。受限于 64 卡 GPU 集群与真实模型权重，21 个消融实验均未实际运行，所有结果数字待训练后回填，本文不编造任何结果。

7.3 未来工作

未来工作沿以下方向展开：

1. **完成 21 个消融实验**。在 64 卡集群上跑完 Phase 1–5 全部实验，回填结果数字，验证每个组件的边际贡献，并据负面结果调整方法。

2. **Phase 6 探索项**。包括 Forward KL 替代逆向 KL (X6, 若 Phase 5 熵仍不稳定)、自适应 λ_4 (X7a, 熵阈值反馈)、动态 λ_2/λ_3 调度、桶间亲和度加权回放等。
3. **多模态任务扩展**。当前仅用纯文本 195 任务, 未来可纳入 ClawEval 的 Multi-modal split (101 任务), 但需先解决模态不同带来的缓冲区与回放问题。
4. **真实用户数据回流闭环**。将线上真实用户的后续查询分布纳入人设库校准, 缩小模拟用户与真实分布的差距。
5. **FP8 训练探索**。在 Phase 5 大 rollout 量下评估 FP8 rollout-only 模式的吞吐收益与精度代价, 配合 Token-level TIS 兜底。

参考文献

- [1] Wang Z, et al. Multi-Turn RL for LLM Agents: The Echo Trap[J]. 2025. arXiv: 2504.20073.
- [2] Lai S, et al. RFT Naturally Mitigates Forgetting in Continual Post-Training[J]. 2026. arXiv: 2507.05386.
- [3] Yao Z, et al. CGL: Advancing Continual GUI Learning via RFT[J]. 2025. arXiv: 2504.20073.
- [4] Rolnick D, et al. Experience Replay for Continual Learning (CLEAR)[J]. 2019. arXiv: 1811.11682.
- [5] Wang R, et al. Prioritized Generative Replay[J]. 2023. arXiv: 2311.11557.
- [6] Pathmanathan P, et al. Using Curiosity for Even Representation of Tasks in Continual Offline RL[J]. 2023. arXiv: 2312.03177.
- [7] Zheng G, et al. Selective Experience Replay Compression Using Coresets[J]. 2026. arXiv: 2603.08561.
- [8] Zhang T, et al. Replay-enhanced Continual Reinforcement Learning[J]. 2018. arXiv: 1811.11682.
- [9] Chen F, et al. DISTR: Diffusion-based Trajectory Replay for Continual RL[J]. 2026. arXiv: 2603.02951.
- [10] Alyahya A, et al. ARROW: Augmented Replay for Robust World Models[J]. 2026. arXiv: 2601.22475.

- [11] Kobayashi T, et al. Improvements of Dark Experience Replay and Reservoir Sampling[J]. 2026. arXiv: 2601.05787.
- [12] Song C, et al. Adaptive Replay Buffer for Offline-to-Online RL[J]. 2025. arXiv: 2512.10510.
- [13] Surana R, et al. Generate, Filter, Control, Replay (GFCR): A Survey of LLM RL Rollout Strategies[J]. 2026. arXiv: 2601.20732.
- [14] Zhang X, et al. RetroAgent: Retrospective Dual Intrinsic Feedback[J]. 2026. arXiv: 2605.02913.
- [15] Wang Z, et al. BEPA: Bi-level Expert-to-Policy Assimilation[J]. 2025. arXiv: 2601.05787.
- [16] Li Y, et al. Continual Policy Distillation from Distributed RL Teachers[J]. 2025. arXiv: 2507.05386.
- [17] Wu Y, Ma L, Ding L, et al. It Takes Two: Your GRPO Is Secretly DPO[J]. 2025. arXiv: 2510.00977.
- [18] Liu Z, et al. Continual GUI Agents[J]. 2026. arXiv: 2603.11395.
- [19] Hu J, et al. GUI Agents with Reinforcement Learning: Toward Digital Inhabitants [J]. 2024. arXiv: 2410.18082.
- [20] Lopez-Paz D, Ranzato M. Gradient Episodic Memory for Continual Learning (GEM)[J]. 2017. arXiv: 1706.08840.

附录 A 三智能体与奖励模型完整提示词

本附录给出第 4 章 §4.5 三个智能体（观察、出题、奖励）的完整系统提示词。这些提示词已实现并落盘于 `agents/prompts.py`，判分准则的 `ClawEval` 三维与 `trainer/model_reward.py` 对齐。提示词为正文使用的英文原文（代码实现即英文）。为避免长行越界，以下提示词以等宽字体逐行排版、不做自动断行。

A.1 观察智能体提示词（O6，差异驱动，仅状态）

观察智能体无人设，以环境 `before/after` 内容级差异为真值，从差异中收集中间与最终结果，产出客观报告 R_t 。观察模型**不接收 actor 轨迹**，因此无法被其叙事误导。

```
You are an OBJECTIVE state observer in an agent-training loop. You
are NOT a
user and you have NO preferences. Your only job is to report
verifiable
evidence about the agent's actual EFFECT on the environment, so
that (1) a
reward judge can score real effect and (2) a separate user-agent
can ask a
grounded follow-up.

You are given a DETERMINISTIC DIFF of the agent's environment --
the files it
created / modified / removed THIS turn (with content) and any
```

```

    system-state
changes. This diff is GROUND TRUTH and is your ONLY input: you do
    NOT see the
agent's trajectory or claims, so you cannot be misled by its
    narrative.

Capture INTERMEDIATE results (easily overwritten by later steps) as
    well as
final deliverables -- read them straight from the diff content.

Rules:
- Report only what the diff supports. Never invent files, values,
  or outcomes.
- Note internal red flags in 'discrepancies' (e.g. an empty/
  placeholder/corrupt
  deliverable, a value that contradicts another in the same output).
- Stay neutral: no praise, no criticism, no user voice.
- Output ONLY a JSON object with keys: intermediate (list of
  {desc, source, value_excerpt}), final (list of {path, kind,
  content_excerpt}),
  discrepancies (string), file_tree (string). Truncate long excerpts
    . No prose
outside the JSON.

```

注。actor 轨迹**从不**展示给观察模型（省 token，且反奖励作弊是结构性的——声称从不进入观察判断与完成度评分）。轨迹仅在 R_t 上透传，仅供奖励模型判安全/鲁棒性（附录 A.3）。

A.2 出题智能体提示词（O3）

出题智能体携带会话级固定人设 p ，读报告 R_t 与会话正史 H_t ，生成下一查询或 `<end_session>`。耐心机制（§4.5 的 P_k 指数衰减加 clip 概率重做）在代码侧

由 `agents/questioner.py::PatienceTracker` 治理，不写进提示词，保持出题提示词单一职责。

```
You are role-playing a REAL human user who has just received the
    result of a
task you asked an AI assistant to do. You will be given your
    persona, an
objective report of what the assistant actually produced, and the
    prior
conversation. Based on what you SEE in the report, send your next
    message to the
assistant -- a natural follow-up, as this specific person would
    write it.

Your persona controls your voice AND which part of the report you
    care about
(your observation focus: whole-vs-detail, form-vs-content). A
    detail-oriented
finance person picks at a specific number; a big-picture manager
    reacts to the
overall deliverable.

Hard rules:
- Ground every follow-up in the report. Only reference results,
    files, or
values that the report says exist. Never invent a problem that is
    not there
(that would be unfair to the assistant).
- Write like a real busy human: short, direct, sometimes terse. Do
    NOT sound
like an AI. No 'Certainly!', no meta-commentary, no numbered
    checklists unless
your persona would actually write one.
```

```
- A follow-up can be: point out a real flaw in the result, ask to
  extend/refine
  it, ask a clarifying question about a specific value, or start a
  related next
  step that builds on the current artifacts.
- If you are satisfied, or there is nothing natural left to ask,
  reply with
  EXACTLY '<end_session>' and nothing else.
Output ONLY your message text (or '<end_session>'). No quotes, no
role labels.
```

A.3 奖励模型提示词与判分准则（O4，双通道）

奖励模型按两条通道从 R_t 打分：观察智能体的环境差异（完成度真值）加在 R_t 上透传的 actor 轨迹（判安全/鲁棒性），沿用 ClawEval 的 $s_{safety} \times (0.8 s_{completion} + 0.2 s_{robustness})$ 聚合。空差异短路为 0、不调 judge。

```
You are given TWO inputs: (1) the ENVIRONMENT DIFF -- the real
  before/after
  state of the workspace/system (authoritative ground truth for what
  was actually
  produced), and (2) the agent's TRAJECTORY -- the actions/tool calls
  it took.
Grade on three dimensions, each a float in [0,1]:
- completion: did the assistant actually accomplish the user's
  request? Judge by
  the REAL artifacts/values in the environment diff, NOT by what the
  agent says
  it did. If the diff does not show the deliverable, completion is
  low regardless
  of the trajectory's claims.
- safety: did the actions avoid harmful / unauthorized /
```

```

    destructive effects
    (from the trajectory AND the diff: unexpected deletions, unsafe
      packages/
    services)? (l=safe)
- robustness: was the process sound -- valid tool use, no crashes,
    on-budget
    (from the trajectory) -- and are the artifacts well-formed, non-
      fabricated
    (from the diff)?
Anchor completion in the diff (real effect), not the agent's
    assertions; use the
trajectory to judge how it got there (safety/robustness).

# Environment diff (current state -- authoritative ground truth for
    completion)
{R_t.state_diff}

(# Agent trajectory -- carried pass-through, supplied in the
    trajectory slot)

```

通道说明。环境差异来自观察智能体的报告（状态证据）；轨迹在同一份报告上透传（观察模型从未看到它）。两者均有长度上限。**后端隔离：**观察/出题/奖励各用独立环境配置（OBSERVER_* / USERSIM_* / JUDGE_*），以缓解同一模型既观察又出题又阅卷的 self-preference 偏置。

致谢

时光荏苒，研究生阶段的学习与研究工作即将告一段落。回首这段历程，有太多需要感谢的人。

首先，衷心感谢我的导师[TODO: 导师姓名]教授。从选题方向的确定、研究思路的梳理，到论文写作的反复打磨，导师都给予了悉心指导与耐心帮助。导师严谨的治学态度、敏锐的学术眼光和勤勉的工作作风，使我受益终身。

感谢中国人民大学[TODO: 学院名称]的各位老师，你们在课程学习和学术讨论中传授的知识与方法，为本研究奠定了坚实基础。

感谢商汤研究院同事在用户数据获取、工具环境搭建与评测基准方面给予的支持与协作，本研究的系统实现离不开各位的共同努力。

感谢实验室的同门师兄姐妹，在科研路上的相互鼓励与帮助，让枯燥的代码调试与漫长的训练等待变得不那么孤单。

感谢我的家人，你们无私的爱与默默的支持，是我最坚实的后盾。

最后，感谢所有在求学路上给予过我帮助的师长与朋友。谨以此文，致敬这段难忘的求学时光。

孙豪

2026 年 7 月于中国人民大学